Guia de Demos en Vivo

DBSCAN Paralelo con OpenMP — Computo Paralelo y en la Nube

Este documento explica, para cada uno de los 5 scripts de demostracion que viven en la carpeta **demos/** del proyecto, que hace, como fue construido, como funciona internamente paso a paso, y trae un guion de presentacion listo para leer o parafrasear mientras se corre cada uno en vivo.

Los 5 scripts son: **demo1_serial.sh**, **demo2_p1.sh**, **demo3_p2.sh**, **demo4_comparativa.sh** y **demo_rapido.sh** (version corta de respaldo).

Jose David — Computo Paralelo y en la Nube, ITAM, 7mo semestre

Como estan construidos los demos (vista general)

Los 5 scripts son archivos **bash** (extension .sh) que viven dentro de la carpeta **demos/** del proyecto **dbscan_proyecto/**. Todos siguen la misma estructura basica, asi que entender una vez esta mecanica te deja entender los 5:

1. Parametros fijos al inicio del script

Cada script define al inicio variables como `EPS=0.03` (epsilon, el radio de vecindad de DBSCAN), `MIN=10` (minSamples, el minimo de vecinos para que un punto sea nucleo/core), y segun el demo, el tamano de problema `N` o la lista de tamanos a probar.

2. Deteccion automatica del hardware

Los demos 2, 3 y 4 usan `sysctl -n hw.ncpu` (un comando propio de macOS) para detectar cuantos hilos logicos tiene la maquina donde se esta corriendo, en vez de tener un numero fijo escrito a mano. Con ese numero arman automaticamente una lista de hilos a probar: 1, cores/2, cores, y cores*2 — que es exactamente el conjunto de valores que pide el enunciado del proyecto para el experimento de hilos.

3. Ejecutar los binarios y capturar su tiempo

Cada binario (`dbscan_serial`, `dbscan_p1`, `dbscan_p2`) imprime, al terminar, una linea con el formato `TIEMPO version,n,k,hilos,tiempo` por su salida de error estandar (stderr), no por la salida normal (stdout). Los scripts capturan esa linea combinando ambas salidas con `2>&1` y filtrandola con `grep '^TIEMPO'`, y luego la separan en sus 5 campos con `IFS=',' read -r`. De ahi sacan el tiempo exacto de cada corrida.

4. Calcular metricas derivadas con awk

Bash por si solo no hace matematicas con decimales, asi que los scripts usan `awk` como calculadora de bolsillo para sacar: el **speedup** (tiempo serial entre tiempo paralelo), la **eficiencia** (speedup entre numero de hilos, expresado como porcentaje), y el **throughput** (numero de puntos entre tiempo, es decir, puntos procesados por segundo).

5. Interpretacion calculada con los numeros reales de la corrida

Esto es lo mas particular de estos demos: la explicacion que aparece al final de cada tabla **no es texto fijo** escrito de antemano. Cada script guarda los resultados de cada vuelta del ciclo en un arreglo de bash (por ejemplo `RESULTS_T1+=("$hilos $speedup")`), y despues usa `sort` para encontrar cual configuracion dio el mejor y el peor resultado, y arma una frase con esos valores reales. Por eso, si corres el mismo demo dos veces en maquinas distintas (o incluso la misma maquina en momentos distintos), el texto de interpretacion cambia junto con los numeros — no esta inventado.

6. Pausas controladas

Al final de cada tabla o seccion, los scripts llaman a una funcion `pausa()` que simplemente espera 12 segundos (`sleep 12`) antes de seguir. Esto le da tiempo al presentador de explicar cada resultado en voz alta sin que la terminal siga imprimiendo cosas encima.

Demo 1 — Version Serial (demo1_serial.sh)

Que hace

Corre la version **serial** de DBSCAN (sin ningun hilo, sin paralelismo) sobre los 8 tamanos de problema oficiales del proyecto (20,000 hasta 200,000 puntos), y muestra una tabla con el tiempo que tarda cada uno, cuantos puntos procesa por segundo, y cuanto crece el tiempo respecto al tamano anterior.

Como fue construido

Es el demo mas simple de los 4 porque la version serial no tiene parametros de paralelismo (no hay hilos ni k que variar) — el unico eje que tiene sentido explorar es el tamano del problema, n. El script guarda los 8 tamanos en la variable `TAMANOS` y recorre cada uno en un ciclo `for`.

Como funciona paso a paso

- Define epsilon=0.03 y minSamples=10 (los mismos parametros que usa la libreta oficial del curso para esta escala de puntos).
- Por cada tamano n en la lista, verifica que exista el archivo datos/puntos_N.csv (si no existe, lo salta con un aviso en vez de tronar).
- Corre ./bin/dbscan_serial sobre ese archivo, capturando la linea TIEMPO y extrayendo el tiempo de ejecucion.
- Calcula el throughput (n / tiempo) con awk.
- Si ya hay un tiempo anterior guardado en la variable prev, calcula el factor de crecimiento (tiempo actual / tiempo anterior) y lo imprime; si es la primera fila, imprime un guion porque no hay anterior con que comparar.
- Al terminar el ciclo, compara el primer tiempo (n=20,000) contra el ultimo (n=200,000) para calcular cuantas veces mas lento es procesar 10 veces mas puntos, y clasifica ese crecimiento como lineal, mejor que lineal, o peor que lineal segun que tan cerca este del factor 10.

Que significa cada columna de la tabla

Columna	Que es
n	Numero de puntos de entrada en ese archivo CSV
tiempo (s)	Cuanto tardo dbscan_serial en procesar ese archivo completo
throughput (pts/s)	Puntos procesados por segundo (n entre tiempo)
crecimiento	Cuantas veces mas tardo este tamano respecto al tamano anterior en la tabla

Guion de presentacion

(Leelo o parafraseado mientras corres ./demo1_serial.sh)

"Antes de hablar de paralelismo, necesitamos una linea base: que tan rapido resuelve DBSCAN el problema **sin** ningun hilo extra, un solo proceso recorriendo todos los puntos uno por uno. Eso es lo que corre este primer demo: la version serial, probada en los 8 tamanos de problema que pide el proyecto, de 20 mil hasta 200 mil puntos.

Como ven, el tiempo sube conforme crece n — eso es esperado, hay mas puntos que comparar entre si. La columna de throughput nos dice cuantos puntos procesa por segundo, y la de crecimiento nos dice cuanto mas lento se vuelve cada salto de tamano respecto al anterior.

Al final el script compara el tiempo mas chico contra el mas grande y nos dice si el crecimiento es cercano a lineal o no — esto importa porque nos dice que tan bien se comporta el algoritmo segun crece el problema, independientemente de si lo paralelizamos o no. Esta es la version que las siguientes dos demos van a superar por varias veces."

Demo 2 — Primera Version Paralela (demo2_p1.sh)

Que hace

Corre la primera version paralela (**P1**, paralelizacion simple del bucle principal con OpenMP) y muestra dos tablas: una variando el numero de hilos con n fijo en 200,000, y otra variando el tamano n con los hilos fijos en 8. Cada tabla trae su propia interpretacion calculada con los resultados reales de esa corrida.

Como fue construido

P1 si tiene un parametro de paralelismo (el numero de hilos, -t), asi que este demo explora dos ejes por separado para no mezclar variables: primero hilos con n fijo, luego n con hilos fijos. Para la lista de hilos, el script detecta los cores reales de la maquina con `sysctl -n hw.ncpu` y arma la lista 1, cores/2, cores, cores*2 — asi el demo se adapta automaticamente sin importar en que computadora se corra.

Como funciona paso a paso — Tabla 1 (hilos)

- Corre una vez la version serial con n=200,000 para tener el tiempo de referencia contra el cual medir el speedup.
- Por cada valor de hilos en la lista detectada (ej. 1, 5, 10, 20), corre ./bin/dbscan_p1 con -t .
- Calcula el speedup (tiempo serial / tiempo P1) y la eficiencia (speedup / hilos, en porcentaje) con awk.
- Guarda cada resultado (hilos, speedup, eficiencia) en el arreglo RESULTS_T1.
- Al terminar el ciclo, ordena ese arreglo por speedup para encontrar la mejor y la peor configuracion, y arma la frase de interpretacion con esos valores reales usando sort y cut.

Como funciona paso a paso — Tabla 2 (tamano n)

- Recorre los 8 tamanos oficiales de n (20,000 a 200,000).
- Para cada uno, corre tanto la version serial como P1 con hilos fijos en 8, y calcula el speedup y el throughput de P1.
- Guarda el speedup de cada n en el arreglo RESULTS_T2, y al final calcula el minimo y el maximo para decir entre que rango se mantuvo el speedup sin importar el tamano del problema.

Que significa cada metrica nueva

Metrica	Formula	Que nos dice
speedup	tiempo_serial / tiempo_paralelo	Cuantas veces mas rapido es P1 que serial
eficiencia	(speedup / hilos) * 100	Que tan bien se aprovecha cada hilo (100% = ideal)
throughput	n / tiempo	Puntos procesados por segundo

Guion de presentacion

"Ahora vamos con la primera version paralela, que llamamos P1. La idea de P1 es sencilla: en vez de que un solo hilo recorra todos los puntos, usamos OpenMP para repartir ese trabajo entre varios hilos que corren al mismo tiempo.

En la primera tabla dejamos fijo el tamano del problema en 200 mil puntos y vamos subiendo el numero de hilos. Fijense como el speedup sube conforme agregamos hilos, pero no de forma perfecta — la columna de eficiencia nos dice que porcentaje de ese speedup ideal realmente logramos. Nunca llegamos al 100% porque hay un costo de crear los hilos, sincronizarlos, y que compitan por acceso a memoria.

En la segunda tabla hacemos lo contrario: dejamos los hilos fijos y variamos que tan grande es el problema, para confirmar que el speedup de P1 no depende de si el problema es chico o grande — se mantiene estable en todo el rango.

La maquina detecto automaticamente sus propios cores, asi que estos numeros de hilos que ven cambian segun en que computadora se corra esto — por eso el script no trae numeros fijos escritos a mano."

Demo 3 — Segunda Version Paralela (demo3_p2.sh)

Que hace

Corre la segunda version paralela (**P2**, que reparte el trabajo en k bloques en vez de un reparto directo por hilo) y muestra tres tablas: speedup vs. hilos, speedup vs. k (el numero de bloques), y speedup vs. tamano n.

Como fue construido

P2 tiene **dos** parametros de paralelismo: el numero de hilos (-t) y el numero de bloques en que se divide el trabajo (-k). Esto hace que P2 tenga un eje extra que explorar respecto a P1. Durante el desarrollo de estos demos encontramos un detalle importante: si el numero de hilos es mayor que k, algunos hilos se quedan sin bloques que procesar y el rendimiento se cae (lo detectamos comparando una corrida con k fijo en 8 contra hilos de hasta 20 — el tiempo empeoraba en vez de mejorar). Por eso, en la Tabla 1 de este demo, k siempre se calcula como el doble de los hilos (k = hilos * 2), garantizando que nunca falten bloques.

Como funciona paso a paso — Tabla 1 (hilos, con k=hilos*2)

- Corre la version serial una vez para tener el tiempo de referencia.
- Por cada valor de hilos detectado automaticamente, calcula k_usado = hilos * 2 y corre ./bin/dbscan_p2 con -t -k .
- Calcula speedup y eficiencia igual que en P1, y guarda los resultados para encontrar la mejor configuracion al final.

Como funciona paso a paso — Tabla 2 (k, con hilos fijos en 8)

- Recorre el rango completo de k que pide el proyecto: 2, 4, 8, 16, 32, 64.
- Para cada k, corre P2 con hilos fijos en 8 y calcula el speedup contra el serial de referencia.
- Al final, encuentra cual k dio el peor resultado (tipicamente uno chico, donde k < hilos, causando desbalance) y cual dio el mejor.

Como funciona paso a paso — Tabla 3 (tamano n)

- Igual que en el Demo 2, recorre los 8 tamanos oficiales con hilos fijos en 8 y k fijo en 16 (un valor seguro, mayor que los hilos).
- Calcula speedup y throughput para cada tamano, y al final reporta el rango de speedup observado.

Guion de presentacion

"P2 es nuestra segunda estrategia de paralelizacion. En vez de repartir los puntos directamente entre los hilos como hace P1, P2 primero divide todo el trabajo en k bloques, y esos bloques se reparten entre los hilos disponibles. Es una capa extra de control sobre como se distribuye el trabajo.

En la primera tabla variamos los hilos, pero con un detalle importante: hacemos que k siempre sea el

doble de los hilos. Descubrimos durante las pruebas que si hay menos bloques que hilos, algunos hilos se quedan sin nada que hacer y el rendimiento se cae — asi que nos aseguramos de que eso nunca pase.

En la segunda tabla exploramos especificamente el efecto de k, dejando los hilos fijos en 8 y probando el rango completo que pide el proyecto: de 2 hasta 64 bloques. Van a ver que con k chico (como 2 o 4) el rendimiento es malo, pero en cuanto k alcanza o supera el numero de hilos, el rendimiento se vuelve estable sin importar que tan grande sea k despues de eso.

Y en la tercera tabla confirmamos, igual que con P1, que el speedup de P2 se mantiene estable sin importar el tamano del problema."

Demo 4 — Comparativa Serial vs P1 vs P2 (demo4_comparativa.sh)

Que hace

Es el demo mas completo: primero imprime la informacion del hardware y software donde se esta corriendo el experimento (como pide explicitamente el PDF del proyecto), despues compara P1 y P2 cabeza a cabeza contra la version serial con los mismos hilos, y por ultimo corre una **verificacion de correctud** que confirma que las dos versiones paralelas clasifican exactamente los mismos puntos que la version serial — no solo que son mas rapidas, sino que dan el resultado correcto.

Como fue construido

Este demo nacio de una necesidad especifica del proyecto: el PDF pide explicar *por que una version paralela fue mejor o equivalente a la otra*, y tambien pide describir el equipo de hardware y software donde se corrieron los experimentos. Ningun demo anterior comparaba P1 y P2 en la misma tabla, asi que se construyo este cuarto demo especificamente para cerrar ese hueco.

La parte de hardware usa varios comandos propios de macOS: `machdep.cpu.brand_string` (el nombre del chip), `hw.physicalcpu`, `hw.perflevel0/1.physicalcpu` (cores de rendimiento vs. eficiencia, relevante en los chips Apple Silicon), `hw.memsize` (RAM, convertida de bytes a gigabytes con awk), `sw_vers` (version de macOS), y la version del compilador real usada (g++-16, no el g++ generico que en Mac apunta a Clang).

Como funciona paso a paso — Tabla comparativa

- Corre la version serial una sola vez (su tiempo es constante, no depende de los hilos).
- Por cada valor de hilos de la lista detectada, corre P1 y P2 (con k = hilos*2 para P2, igual que en el Demo 3) y calcula el speedup de cada una contra el mismo serial de referencia.
- Guarda los resultados de P1 y P2 en arreglos separados (RESULTS_P1 y RESULTS_P2) para encontrar, al final, con cuantos hilos cada version logro su mejor speedup.

Como funciona paso a paso — Verificacion de correctud

Esta es la parte mas importante del demo desde el punto de vista academico. Cada archivo de salida que generan los binarios (serial, P1, P2) trae una columna llamada `tipo` que clasifica cada punto en 3 categorias: **0 = ruido**, **1 = nucleo/core**, **2 = borde**. El script define una funcion `conteo_tipo()` que usa awk para contar cuantos puntos cayeron en cada categoria dentro de un archivo CSV de salida.

- Corre P1 y P2 una vez mas con una configuracion fija y conocida (t=8, k=16), guardando sus salidas.
- Cuenta ruido/core/borde en la salida de serial, de P1 y de P2 por separado usando conteo_tipo().
- Compara los 3 conteos con una cadena de condiciones if: si los 9 numeros (3 categorias x 3 versiones) coinciden exactamente, imprime una marca de visto bueno en verde; si cualquiera no coincide, imprime una alerta en rojo.

Por que esto importa para la nota del proyecto

Un error comun es demostrar solo la velocidad de las versiones paralelas sin probar que su resultado sigue siendo correcto. Una paralelizacion mal hecha puede ser muy rapida y dar clusters equivocados (por ejemplo, por condiciones de carrera al escribir en memoria compartida). Esta verificacion demuestra, con datos reales de la corrida, que eso no esta pasando aqui.

Guion de presentacion

"Este ultimo demo junta todo. Primero les mostramos el hardware real donde estamos corriendo esta demostracion en vivo — el chip, cuantos cores de rendimiento y de eficiencia tiene, la RAM, la version de macOS y del compilador. Esto es relevante porque los numeros que van a ver aqui pueden ser distintos a los del reporte escrito, que se corrio en otra maquina con otro numero de cores.

Despues corremos P1 y P2 lado a lado, con exactamente los mismos hilos y el mismo tamano de problema, para compararlas directamente. Van a ver que ambas estrategias de paralelizacion — reparto directo en P1, bloques en P2 — dan resultados de velocidad muy parecidos entre si.

Pero lo mas importante viene al final: la verificacion de correctud. No basta con que las versiones paralelas sean rapidas, tambien tienen que dar el **mismo resultado** que la version serial. Aqui contamos cuantos puntos quedaron clasificados como ruido, como nucleo, y como borde en cada una de las 3 versiones, y confirmamos que los tres conteos coinciden exactamente. Eso nos dice que paralelizar el algoritmo no cambio el resultado — solo lo hizo mas rapido, que es justo lo que se busca al paralelizar."

Demo rapido — Version corta de respaldo (demo_rapido.sh)

Que hace

Es un script envoltorio (wrapper) que simplemente llama, en orden, a demo1_serial.sh y a demo4_comparativa.sh. No tiene logica propia — su unico trabajo es encadenar esos dos demos para cubrir lo esencial (linea base serial, comparativa de ambas paralelas, y la verificacion de correctud) en caso de que el tiempo de presentacion sea corto.

Como fue construido

Se escribio como respaldo despues de notar que correr los 4 demos completos (con sus pausas de 12 segundos entre tablas) puede tomar varios minutos. Si el profesor da poco tiempo para la demostracion en vivo, este script permite mostrar lo mas importante sin tener que recordar en que orden correr los otros dos manualmente.

Codigo completo (son solo 9 lineas)

```
#!/bin/bash
echo
echo "=== VERSION RAPIDA: corre Demo 1 (serial) y Demo 4 (comparativa + correctud) ==="
echo " Si necesitas los detalles de P1 o P2 por separado, corre demo2_p1.sh / demo3_p2.sh
aparte."
echo

./demo1_serial.sh
./demo4_comparativa.sh
```

Guion de presentacion

> "Si el tiempo apremia, con este unico comando corremos lo esencial: la linea base serial y la comparativa final con la verificacion de correctud — que entre los dos ya responden las preguntas clave del proyecto: que tan rapido es cada version, y si el resultado paralelo sigue siendo correcto."

Nota final: todos los scripts guardan sus archivos de salida (clasificacion de puntos) dentro de la carpeta `datos/`, con nombres que identifican la version, el tamano n, los hilos o la k usada — por si se necesita revisar o graficar algun resultado especifico despues de la presentacion.